

Here is a polished **Azure DevOps for QA – Overview** section suitable for a training plan or course curriculum:

Azure DevOps for QA – Overview

Azure DevOps for QA – Overview

Azure DevOps Test Management

Topics Covered

- **Test Management in Azure DevOps**
 - Understanding Azure DevOps Test Plans
 - Test Plans and Test Suites
 - Organizing tests by feature, module, or sprint
- **User Stories**
 - Understanding User Stories and acceptance criteria
 - Linking test cases to User Stories
 - Establishing requirements-to-test traceability
- **Test Cases**
 - Creating manual test cases
 - Defining test steps, expected results, and test data
 - Adding test cases to test suites
 - Linking test cases to requirements
- **Test Suites**
 - Requirement-based suites
 - Query-based suites
 - Static test suites
 - Organizing test cases for execution
- **Test Execution**
 - Running manual test cases
 - Recording Pass, Fail, Blocked, and Not Applicable results
 - Capturing execution notes and tester comments
 - Reviewing test execution history
- **Defect Logging**
 - Creating bugs from failed test cases
 - Defining severity and priority
 - Adding reproduction steps
 - Assigning defects to the appropriate team member
- **Linking Defects to Tests**
 - Linking Bugs with Test Cases

- Understanding work-item relationships
- Maintaining end-to-end traceability between requirement → test → defect
- **Evidence**
 - Attaching screenshots and test evidence
 - Adding automation logs
 - Attaching Playwright traces, videos, screenshots, and reports
 - Creating evidence-backed defects
- **Retesting**
 - Executing tests after defect resolution
 - Recording retest results
 - Updating defect status
 - Regression testing after fixes
- **Queries**
 - Creating basic work-item queries
 - Finding failed tests and active bugs
 - Filtering by state, priority, severity, assignee, or sprint
- **Basic QA Dashboards**
 - Test execution status
 - Defect status
 - Open vs. resolved defects
 - Test coverage and execution trends
 - Using widgets for QA reporting
- **Traceability**
 - User Story → Test Case → Test Execution → Bug
 - Understanding end-to-end QA traceability
 - Using links and work-item relationships for auditability

Hands-on Exercise

Scenario: Diagnose an Automated Test Failure and Raise a Defect

Participants will:

1. Execute a sample **Playwright automated test**.
2. Analyze the failed test and identify the actual failure.
3. Review the Playwright error message, screenshot, trace, or video.
4. Determine whether the failure is an **application defect, test automation issue, or test-data/environment issue**.
5. Navigate to the relevant User Story in Azure DevOps.
6. Create an Azure DevOps **Bug** with:
 - Title

- Description
- Environment
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Assigned team member

7. Attach the relevant **automation evidence**.
8. Link the Bug to the failed Test Case/User Story.
9. Update the Test Case execution result.
10. Retest after the defect is resolved.
11. Update the Bug and Test Case with the final outcome.
12. Verify end-to-end **requirements** → **test** → **execution** → **defect traceability**.

Suggested Practical Scenario

User Story:

As a customer, I should be able to log in using valid credentials.

Test Case:

Verify that a valid user can successfully log in.

Automation:

Playwright test executes the login scenario.

Failure:

The application displays an incorrect error message after valid credentials are submitted.

Participant Task:

Diagnose the failure, collect the Playwright screenshot/trace, create an evidence-backed Bug in Azure DevOps, link it to the Test Case/User Story, and perform retesting after the fix.

This can also be expanded into a **2–3 hour hands-on Azure DevOps QA workshop** with a step-by-step exercise covering **User Story** → **Test Case** → **Test Suite** → **Playwright execution** → **Bug** → **Evidence** → **Retest** → **Traceability**.

